

Typist-ML

Technical Design Document

Decisions, stack, architecture, data model, UML, flows, model setup and build plan for a typing-coach experiment that runs entirely on one MacBook.

Field	Value
Owner	Allen Varghese
Version	1.0, 24 September 2026
Status	Ready to build, pending owner review
Based on	Typist-ML: Project Guide (24 Sep 2026) and planning decisions made the same day
Team	One developer

Contents

1. Summary and what changed
2. Decision log
3. Experiment design (within-person)
4. Who uses it and where it runs
5. Tech stack
6. System architecture
7. Repository layout
8. Database design
9. UML: classes and states
10. Sequences and data flow
11. User flows and screens
12. Telemetry specification
13. Drill generation and model setup
14. API specification
15. Analysis and dashboard
16. Milestones and tasks
17. Engineering standards
18. Risks
19. Research notes and resources
20. Open items and day-one checklist

1. Summary and what changed

This document turns the Typist-ML Project Guide (24 Sep 2026) into a buildable technical design. It records every decision needed to start, the tech stack and model setup, the system architecture, the database schema, UML and sequence diagrams, data and user flows, and a milestone plan with tasks and acceptance criteria. It is written so that a developer who has not seen the planning conversations can pick it up and start on milestone 0.

The research question is unchanged: do targeted drills beat plain practice, and does a small on-device language model write better drills than a dictionary heuristic or a Markov chain? What changed is how the experiment answers it, because the pilot is now one to two people instead of six to ten.

Changes from the Project Guide

Topic	Project Guide said	This design	Reason
Participants	Builder plus 5 to 10 friends	Builder plus at most 1 other person	Decided on 24 Sep 2026
Unit of comparison	Participants assigned to arms (1 to 3 per arm)	Every participant runs all four arms at once, each on a different set of their weak bigrams	With 1 to 2 people, one person per arm makes person and arm impossible to separate
Control	Random-text control arm	A control bigram set that gets no targeted drills, plus a random-text practice block of equal length	Controls for plain practice and for regression to the mean
Held-out split	Suggested, undecided	Adopted and generalised: the control set is the held-out set	Gives a within-person comparison
Participant access	Open	Local-only on the MacBook Pro; the second person types on the same laptop	No hosting, no model serving, data stays on one machine
Browser to storage	Open (local server or desktop wrapper)	Local FastAPI server (Python) that owns SQLite and the ML code	Python is the chosen ML language; simplest local setup
Drill timing	Not specified	Drills are pre-generated in batch by a worker, never in the typing loop	LLM calls take seconds and can be rejected and retried
Live mistake nudges	Not in guide	Deferred to after the pilot	A nudge is an extra intervention that would muddy the comparison
Keyboard and text	Not specified	English, QWERTY, lowercase a to z and space	Keeps bigram tracking simple for v1

Honesty rule carried over: the Project Guide's rules still apply. No WASM claim, no improvement claim until real numbers exist, results are reported as descriptive per-participant changes, and if the heuristic or the control does as well as the LLM, that is the finding.

How to use this document

- **Section 2** is the decision log. If you are unsure why something is built a certain way, look there first.
- **Sections 3 and 4** explain the experiment and who uses the system. Read them before writing any telemetry or generator code, because they define what must be measured.

- **Sections 5 to 11** are the build reference: stack, architecture, repository layout, schema, UML, flows.
- **Sections 12 and 13** are the two specs that matter most for correctness: how keystrokes become bigram statistics, and how drills are generated and validated.
- **Section 16** is the task plan. Start at milestone 0.

2. Decision log

Each decision has an ID so commits, tickets and later changes can refer to it (for example ADR D07). Changing a decision means adding a row with the new choice and the date, not editing the old row.

ID	Decision	Choice	Why	Revisit if
D01	Experiment unit	Within-person. Each participant's weak bigrams are split into 4 sets; each set is assigned one arm	Only 1 to 2 participants	8 or more participants join
D02	Control	Control bigram set with no targeted drills, plus a random-text block of equal length each session	Separates targeting from plain practice and regression to the mean	Never for this pilot
D03	Arms	control, heuristic, markov, llm (as in the guide)	Unchanged research question	Milestone 3 slips (see cut-lines)
D04	Assignment	Rank weak bigrams, deal into sets in snake order, map sets to arms with a seeded shuffle, freeze before practice	Balances weakness across sets and removes builder choice	Never mid-pilot
D05	Outcomes	Primary: change in pooled error rate per set minus change in the control set. Key secondary: same for median inter-key interval	Error rate is the guide's primary; timing is far more sensitive with small counts	Before retest only, by editing the analysis plan
D06	Deployment	Local-only on the MacBook Pro (M1 Pro). The second participant uses the same laptop	No hosting or model serving; same keyboard for everyone	Second person cannot come to the laptop (see section 4)
D07	Frontend	Vite 8, React 19, TypeScript (strict), pnpm, Biome, Vitest	Fast, current, small tooling surface	-
D08	Backend	Python 3.12, uv, FastAPI, SQLAlchemy, Alembic, SQLite in WAL mode, Typer CLI, ruff, pyright, pytest	Python is the chosen ML language; one language for stats, generation and API	-
D09	Model runtime	mlx-lm Python API, version pinned in uv.lock	Direct control of sampling, seeds and tokens-per-second stats; Apple Silicon native	Moving off Apple Silicon (fall back to llama.cpp)
D10	Model	Default mlx-community/Llama-3.2-1B-Instruct-4bit. Short bake-off against Qwen3-1.7B (4-bit) before the pilot, then frozen	Matches the guide's ~1B framing, text-only, well supported	Bake-off shows another model clearly accepts more drills
D11	Generation timing	Batch, via a separate worker process. Keep 3 days of ready drills per arm	LLM latency and rejections never block typing	-
D12	Live nudges	Not during the pilot; shown in the post-retest debrief	Would be an extra intervention	After the pilot (v2)
D13	Text scope	English, QWERTY, lowercase a to z and space, no punctuation. Tracked bigrams are letter-letter only	Keeps attribution simple	v2
D14	Error definition	First-attempt substitution at position j, counted only when position j-1 was correct on first attempt. Backspace allowed	Clean bigram attribution	-

ID	Decision	Choice	Why	Revisit if
D15	Timing	Timestamps from event.timeStamp. IKI = press-to-press. Also store dwell and release-to-press (can be negative)	Rollover typing is common, so release-to-press is unreliable as the main timing	-
D16	Fairness gate	All targeted arms pass one shared validator: length, charset, density range, per-bigram coverage, repetition limits	Arms differ only in method	-
D17	Word list	wordfreq English list, filtered to a to z words	Frequency-ranked, easy to install	License concerns (cite CC BY-SA data in README)
D18	Blinding	Blocks are labelled Block 1 to 4. The second participant is not told the mapping. The builder is flagged as unblinded	Reduces expectation effects	-
D19	No cloud, no WASM	Carried over from the guide	Guide honesty rules	-
D20	Pre-registration	Commit docs/analysis-plan.md with the date before the first practice session	Stops outcome-shopping after seeing data	-

3. Experiment design (within-person)

3.1 Why the design changed

The guide assigned each participant to one arm. With one or two participants, each arm would have zero or one person, so any difference between arms would be a difference between people. The fix is to make the comparison inside each person: each participant's weak bigrams are split into four matched sets, and each set gets a different arm. The same person, on the same days, with the same keyboard, is compared with themselves.

This also answers the guide's open question about a drilled versus held-out split. The control set is the held-out set, so the design adopts that idea and extends it to all four arms.

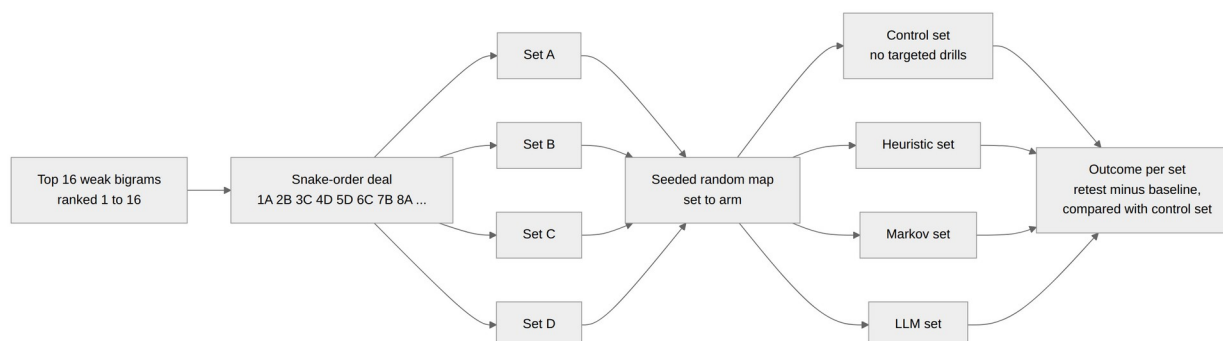


Figure 1. Weak bigrams are dealt into four balanced sets and each set is assigned one arm.

3.2 Schedule per participant

Phase	When	What happens	Length
Baseline	Day 0 and day 1	Two sessions of fixed test text (form A) covering the ~150 most common English letter bigrams	About 10 minutes each
Assign and freeze	End of day 1	Rank weak bigrams, split into sets, map sets to arms, freeze. Worker pre-generates 3 days of drills	Automatic, a few minutes
Practice	Days 2 to 11 (default N = 10)	One session per day: four blocks, one per arm, equal characters, order rotated daily	About 12 to 15 minutes
Retest	Day 12 and day 13	Two sessions of parallel test text (form B) with at least 40 attempts per tracked bigram	About 10 minutes each
Debrief	After retest	Show weak bigrams, changes, and the arm mapping	5 minutes

Rule: baseline must finish no more than 3 days before the first practice session. If practice is delayed longer, redo the baseline. Missed practice days are allowed but logged; the retest happens after N completed practice sessions, not N calendar days.

3.3 Practice session structure

- Each practice session has **four blocks**, one per arm. Each block is **600 characters** by default (two drills of about 300 characters).
- The **control block** is frequency-weighted random words. It targets nothing, so it provides equal practice time without targeting.
- Block order follows a **4 by 4 Latin square** rotated by day, so no arm is always first (warm-up) or last (fatigue).

- Blocks are shown as Block 1 to Block 4. The UI never names the arm.
- Fixed characters per block, not fixed time, so every arm gets the same exposure. Time per block is recorded.

3.4 Choosing the weak bigrams

1. Candidates are letter-letter bigrams from the ~150 most common in English that have at least 20 first-attempt observations in the two baseline sessions.
2. For each candidate, compute a smoothed error rate: posterior mean of a Beta prior updated with the bigram's errors and attempts. The prior is centred on the participant's overall bigram error rate with strength 20 (empirical Bayes). This stops a bigram with 1 error in 3 attempts from ranking first.
3. Rank by smoothed error rate. Break ties with the bigram's median inter-key interval z-score within the participant (slower ranks weaker).
4. Take the top 16 (4 per set). If fewer than 16 qualify, take 12 (3 per set) and log it.
5. Deal them into sets A, B, C, D in snake order: rank 1 to A, 2 to B, 3 to C, 4 to D, 5 to D, 6 to C, 7 to B, 8 to A, and so on. This balances weakness across sets.
6. Shuffle the arm list with a seed stored in `experiment_config` and map A to D onto the shuffled arms. Freeze by writing `frozen_at`. Frozen rows are never updated.

3.5 Outcomes and analysis

Outcome	Definition	Role
Set error rate change	For each set, pooled errors divided by pooled attempts over its bigrams, retest minus baseline	Primary
Targeting effect	Set error rate change minus the control set's change (difference in differences)	Primary comparison
Set IKI change	Median inter-key interval over the set's bigrams, retest minus baseline, minus the control set's change	Key secondary
Overall WPM and accuracy	Session-level, baseline versus retest	Check that speed was not lost elsewhere
Instrumentation overhead	Handler time and dispatch lag per keystroke, p50, p95, p99	Engineering claim
LLM generation	Tokens per second, acceptance rate, attempts per accepted drill	Engineering claim
Drill quality	Target density, mean word length, real-word ratio per arm	Checks fairness between arms

Method. For each set and phase, report errors, attempts and a 95% interval from a Beta posterior (Jeffreys prior). For the targeting effect, draw samples from the four posteriors and report the median effect and the share of samples below zero. This is a description of one or two people, not a significance test. The write-up states the number of participants, sessions and keystrokes plainly.

Known weakness, stated up front: error rates on single bigrams are low (often under 5%). With about 200 attempts per set at retest, a set may show only around 10 errors, so error-rate changes will be noisy. That is why timing (IKI) is the key secondary outcome and why the retest guarantees at least 40 attempts per tracked bigram. Other limits: bigrams share fingers and keys, so practice on one set can spill over to another (this makes effects look smaller, not larger); Markov drills use pseudo-words while tests use real words; and the builder is not blind.

4. Who uses it and where it runs

Everything runs on one MacBook Pro (M1 Pro). The browser, the API, the worker, the database and the model all live on that machine. Nothing is hosted and no model has to be served over the internet.

4.1 The second participant

The recommended setup is that the second person types on the same laptop. That keeps the keyboard constant, keeps all data on one machine, and needs no network setup. It costs them a daily visit to the laptop, so pick someone you see most days.

Option	How	Pros	Cons	Status
A. Same laptop	They open the app on your MacBook and pick their alias	Zero setup, same keyboard, data never leaves the machine	They need daily access to your laptop	Default for the pilot
B. Their laptop, private network	Bind the API and web app to a private tunnel (for example Tailscale) while the pilot runs	They type from anywhere	Setup for them, your laptop must be on, their keyboard differs from yours (fine, because comparisons are within-person)	Optional add-on, not in the v1 plan
C. Hosted web app	Deploy web and API, keep the worker on your laptop	Most convenient for them	Hosting, auth, data leaves your machine, more to build	Out of scope

Because drills are pre-generated, option B would only need the web app and API reachable; the model never has to be exposed. If you switch to B later, only the network binding changes. The architecture does not.

4.2 Consent and privacy

- The consent screen says what is recorded (every key press and release with timestamps, while the typing surface is focused), where it is stored (one SQLite file on your laptop), and that they can ask to delete it at any time.
- Participants are stored by alias only. No name, email or free text is stored.
- The claim in the README becomes: typing data is stored only on the builder's machine and never sent to a third party or a cloud AI service.
- A `typist participant delete --pid` command removes every row for a participant.

5. Tech stack

Versions below are current as of September 2026. At milestone 0, record the exact versions that were installed; the lockfiles (pnpm-lock.yaml, uv.lock) are the source of truth after that. Do not upgrade the model runtime or the model during the pilot.

5.1 Frontend (apps/web)

Concern	Choice	Why	Rejected
Build tool	Vite 8 (Rollup bundler)	Current default for React SPAs; fast dev server	Next.js (server features not needed)
UI	React 19 + TypeScript, strict mode	Guide choice; typed event pipeline	Monkeytype fork (rejected in guide)
Package manager	pnpm	Fast, strict dependency resolution	npm, yarn
Server state	TanStack Query	Caching and retries for API calls	Hand-rolled fetch state
Styling	Tailwind CSS v4	Quick, consistent styling for a small UI	CSS-in-JS
Charts	Recharts	Simple React charts for the dashboard	D3 directly (more work)
API types	openapi-typescript, generated from FastAPI's OpenAPI schema	One contract, no hand-copied types	Manual interfaces
Lint and format	Biome	One fast tool instead of ESLint plus Prettier	ESLint + Prettier
Tests	Vitest (unit), Playwright (one end-to-end smoke test)	Vite-native; real keyboard events in e2e	Jest

5.2 Backend and ML (services/api)

Concern	Choice	Why	Rejected
Language	Python 3.12, managed with uv	Chosen ML language; uv handles Python versions, venvs and locking	pip + venv, Poetry
Web API	FastAPI + Uvicorn, bound to 127.0.0.1	Typed, auto OpenAPI schema, small	Flask, Django
ORM	SQLModel (on SQLAlchemy 2)	Pydantic plus SQLAlchemy in one model class	Raw SQL everywhere
Migrations	Alembic, from day 1	Schema will change during milestones	create_all only
Database	SQLite, WAL mode, foreign_keys on	One file, local, concurrent reader and writer	Postgres (needless for local)
CLI	Typer	Admin commands: assign, analyze, export	argparse
Config	pydantic-settings, .env file	Typed configuration	Globals

Concern	Choice	Why	Rejected
Analysis	numpy, pandas, scipy; matplotlib for README figures; Jupyter for exploration	Standard Python data stack, good for learning	Polars (fine, but less common in tutorials)
Model runtime	mlx-lm (Apple MLX), pinned	Native on Apple Silicon; Python API gives sampler, seed and tokens-per-second control	Ollama (less control), LM Studio (GUI), llama.cpp (fallback only)
Word frequencies	wordfreq	Ranked English word list with frequencies	Scraped lists
Quality	ruff (lint + format), pyright (types), pytest, hypothesis (property tests)	Fast and strict	flake8, black, mypy (fine alternative)

5.3 Repository tooling

- **just** as the task runner (`just dev`, `just test`, `just lint`, `just types`). One command starts the web app, API and worker.
- **pre-commit** runs ruff and Biome before each commit.
- **GitHub Actions** runs lint, type checks and tests on every push. Useful even solo, and it shows up on the repository.
- **Conventional commits** (`feat:`, `fix:`, `docs:`) and small pull requests into main, even when reviewing your own work.

6. System architecture

Three processes run on the laptop. The browser app records keystrokes and shows text. The API process owns the database and computes statistics. The worker process owns the language model and generates drills. They share one SQLite file.

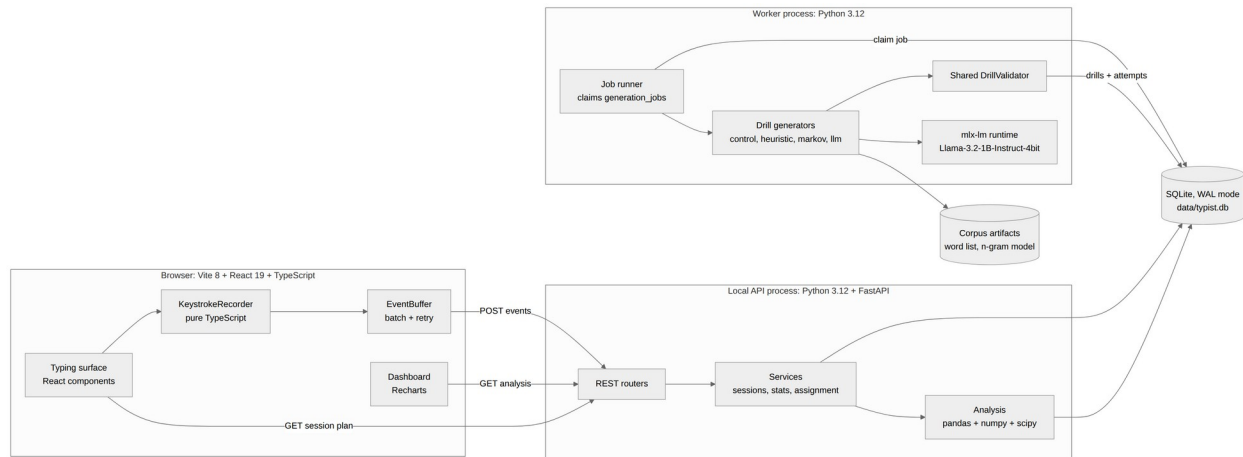


Figure 2. Component view. Only the worker loads the model.

6.1 Component responsibilities

Component	Owns	Must not
Typing surface (React)	Rendering text, cursor, error highlight, block flow	Compute bigram statistics or talk to the model
KeystrokeRecorder (TypeScript)	Capturing keydown and keyup, timestamps, alignment to the target text, overhead measurement	Depend on React; it is a plain class so it can be unit tested
EventBuffer (TypeScript)	Batching events, retries, final flush on unload	Drop events silently
API routers (FastAPI)	HTTP validation and mapping to services	Contain business logic
Services (Python)	Session planning, bigram attribution, assignment, test composition, analysis	Import the model runtime
Worker (Python)	Claiming generation jobs, running generators, validating, saving drills and attempts	Serve HTTP
Generators (Python)	Turning a DrillSpec into candidate text	Read or write the database directly; the GenerationService does that
SQLite	All persistent state	-

6.2 Key architecture decisions

- **One source of truth for statistics.** Bigram attribution runs only in Python, from raw keystroke rows. The browser computes only what it needs for live display (current WPM, error highlight). This means stats can be recomputed after a bug fix with `typist stats recompute`.
- **Raw data is append-only.** `keystroke_events` rows are never updated after insert, except to fill in the key-up time. Everything else is derived and can be rebuilt.

- **Separate worker.** The 4-bit 1B model needs roughly 1 GB of memory and several seconds per drill. Keeping it out of the API keeps request latency flat, and a model crash does not take the API down.
- **Jobs through the database.** The worker polls `generation_jobs` every few seconds and claims a job with one atomic `UPDATE ... WHERE status = 'pending'`. No Redis or Celery is needed for one machine.
- **Localhost only.** The API binds to 127.0.0.1 and allows the Vite dev origin through CORS. In the built version FastAPI serves the static web files, so there is one URL.
- **Everything that affects results is recorded.** Seeds, generator versions, model id, prompt version and thresholds live in `experiment_config` and on each drill row.

7. Repository layout

A single repository with two apps. Folder names are part of the design: juniors should be able to guess where code lives.

```

typist-ml/
  README.md          # what, why, results (numbers only once they exist)
  justfile           # dev, test, lint, types, migrate, worker
  .pre-commit-config.yaml
  .github/workflows/ci.yml
  docs/
    design/          # this document
    analysis-plan.md # committed and dated before first practice (D20)
    decisions/        # one short ADR file per changed decision
  apps/web/
    src/
      app/           # main.tsx, routes.tsx, providers
      telemetry/     # KeystrokeRecorder.ts, TextCursor.ts, EventBuffer.ts,
                     # overhead.ts, types.ts, __tests__/
      features/
        session/     # SessionPage, TypingSurface, BlockIntro, SessionSummary
        onboarding/  # ConsentPage, ParticipantPicker
        admin/       # AdminPage, JobsTable, DataChecks
        dashboard/   # Overview, ParticipantResults, charts
      api/           # client.ts, schema.d.ts (generated)
      e2e/           # Playwright smoke test
  services/api/
    pyproject.toml  uv.lock  alembic.ini  alembic/
    src/typist/
      main.py       # FastAPI app factory
      config.py     # pydantic-settings
      db.py         # engine, session dependency, SQLite pragmas
      models/       # SQLAlchemy table classes (one file per table group)
      schemas/      # request/response DTOs (never return table models)
      api/routers/  # participants, sessions, events, analysis, admin
      services/
        session_planner.py # builds each session's blocks
        attribution.py     # keystrokes -> bigram stats (section 12)
        assignment.py      # ranking, snake split, freeze
        test_composer.py   # baseline and retest texts
        analysis.py        # outcomes and intervals
      generation/
        base.py           # Arm, DrillSpec, DrillCandidate, DrillGenerator
        validator.py
        control.py  heuristic.py  markov.py
        llm/ runtime.py  prompts.py  generator.py
        service.py     # GenerationService: retries, logging, saving
      corpus/          # build.py, word_index.py, ngram.py
      worker.py        # job loop
      cli.py           # typer commands
      tests/ unit/ integration/ fixtures/
  notebooks/          # 01_baseline_eda, 02_model_bakeoff, 03_results
  data/              # gitignored: typist.db, corpus artifacts

```

Shared fixture: `services/api/tests/fixtures/golden_session.json` holds a hand-checked keystroke log and its expected bigram stats. The Python attribution tests and the TypeScript TextCursor tests both use it, so the two languages cannot drift apart on what counts as an error.

8. Database design

SQLite with PRAGMA journal_mode=WAL, foreign_keys=ON and synchronous=NORMAL. Ids are UUID strings except for the two high-volume tables, which use integer ids. Timestamps are UTC ISO-8601. Durations are milliseconds stored as REAL. Every schema change is an Alembic migration.

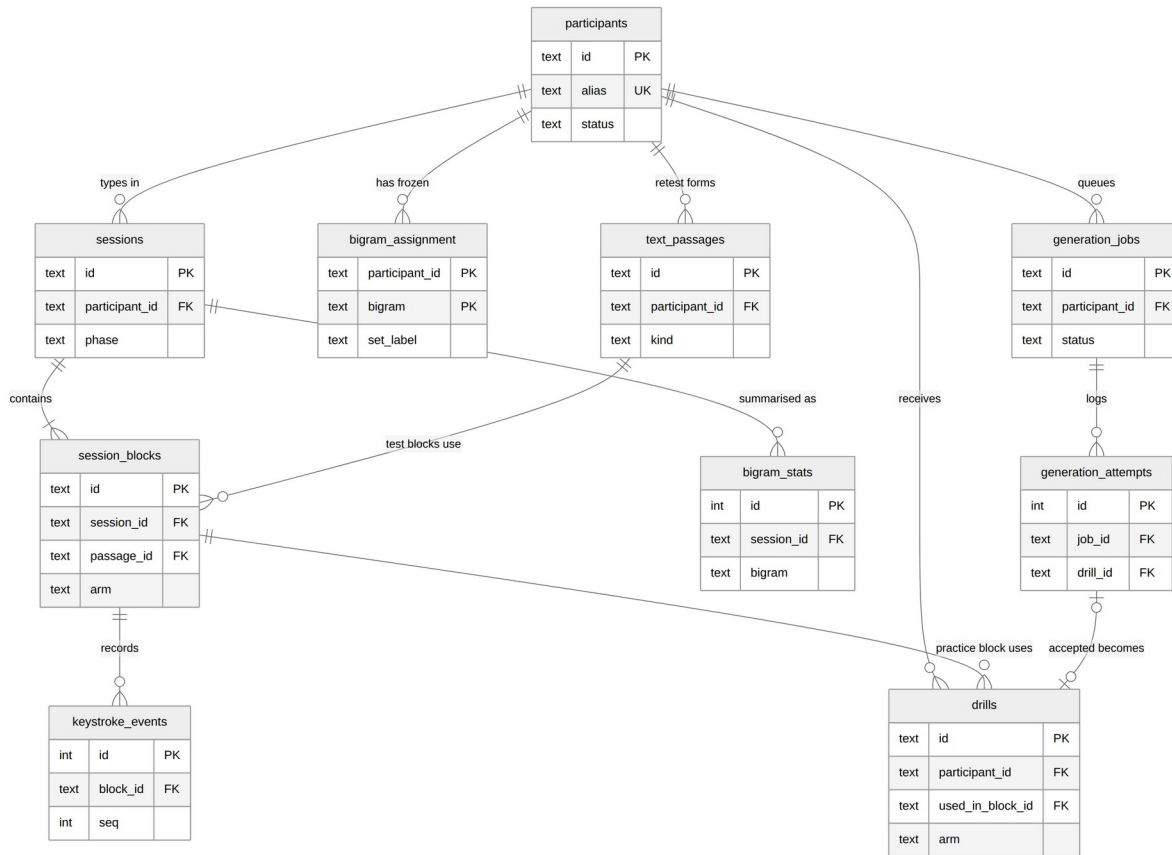


Figure 3. Entity relationships (keys only; full columns below).

8.1 Tables

participants

Column	Type	Notes
id	TEXT PK	UUID
alias	TEXT UNIQUE	Shown in the UI; no real names
is_builder	BOOLEAN	True for you; used to flag unblinded data
is_blinded	BOOLEAN	False for the builder
status	TEXT	enrolled, baseline, assigned, practice, retest, done, withdrawn (see state diagram)
keyboard_label	TEXT	For example "MBP M1 Pro built-in"
consent_at, created_at	DATETIME	

experiment_config

Column	Type	Notes
key	TEXT PK	For example assignment_seed, practice_days, block_chars, d_min, d_max, model_id, prompt_version, sampler
value	JSON	
frozen_at	DATETIME NULL	Set before the first practice session; frozen keys cannot change

text_passages

Column	Type	Notes
id	TEXT PK	
kind	TEXT	baseline or retest
form	TEXT	A (baseline) or B (retest)
part	INTEGER	Order within a session
participant_id	TEXT FK NULL	Null for shared baseline texts; set for retest texts composed after freezing
text	TEXT	
bigram_counts	JSON	Count of each letter bigram in the text, for coverage checks

sessions

Column	Type	Notes
id	TEXT PK	
participant_id	TEXT FK	
phase	TEXT	baseline, practice, retest
day_index	INTEGER	1-based count within the phase
block_order	TEXT	Latin-square row used, for example "LCHM"
status	TEXT	in_progress, completed, abandoned
wpm, accuracy	REAL	Computed on completion
client_info	JSON	Browser, user agent, screen size
started_at, ended_at	DATETIME	

session_blocks

Column	Type	Notes
id	TEXT PK	
session_id	TEXT FK	
block_index	INTEGER	0 to 3
arm	TEXT	test, control, heuristic, markov, llm

Column	Type	Notes
(drills)	-	Practice drills link back through drills.used_in_block_id; a block holds two drills
passage_id	TEXT FK NULL	Test blocks
target_text	TEXT	Exact text shown, stored for reproducibility
started_at, ended_at	DATETIME	

keystroke_events

Column	Type	Notes
id	INTEGER PK	
block_id	TEXT FK	UNIQUE (block_id, seq) makes uploads idempotent
seq	INTEGER	Order within the block, from 0
key, code	TEXT	KeyboardEvent.key (character) and .code (physical key)
t_down_ms, t_up_ms	REAL	event.timeStamp relative to block start; t_up may be null if the page closed
dwel_ms	REAL	t_up minus t_down for this key
iki_ms	REAL	t_down minus previous keystroke's t_down (press-to-press)
release_to_press_ms	REAL	t_down minus previous key's t_up; negative under rollover
target_index	INTEGER	Cursor position when the key was pressed
expected_char	TEXT	Target character at that position
outcome	TEXT	correct, error, backspace
first_attempt	BOOLEAN	True if this is the first keystroke at this target position
handler_ms, dispatch_lag_ms	REAL	Instrumentation overhead (section 12.5)

bigram_stats

Column	Type	Notes
id	INTEGER PK	UNIQUE (session_id, bigram)
participant_id, session_id	TEXT FK	
bigram	TEXT	Two lowercase letters
attempts, errors	INTEGER	Per the rules in section 12.3
iki_median_ms, iki_n	REAL, INTEGER	Only clean, correct pairs; pauses over 2000 ms excluded

bigram_assignment

Column	Type	Notes
participant_id, bigram	TEXT, composite PK	
set_label	TEXT	A, B, C or D
arm	TEXT	Arm mapped to this set

Column	Type	Notes
baseline_rank, weakness_score	INTEGER, REAL	From section 3.4
baseline_attempts, baseline_errors	INTEGER	Snapshot at freeze time
frozen_at	DATETIME	Rows are immutable after this

drills

Column	Type	Notes
id	TEXT PK	
participant_id	TEXT FK	
arm	TEXT	control, heuristic, markov, llm
target_bigrams	JSON	Empty for control
text	TEXT	
target_density, mean_word_len, real_word_ratio	REAL	Quality and fairness metrics
generator_version, model_id, prompt_version, seed	TEXT, INTEGER	Reproducibility
status	TEXT	ready, used, retired
used_in_block_id	TEXT NULL	
created_at	DATETIME	

generation_jobs and generation_attempts

Column	Type	Notes
jobs.id, participant_id, arm	TEXT	
jobs.requested	INTEGER	Number of drills to produce
jobs.status	TEXT	pending, running, done, failed
jobs.retries, error, created_at, started_at, finished_at		
attempts.id, job_id, drill_id	INTEGER, TEXT, TEXT NULL	drill_id set when accepted
attempts.raw_output, clean_text	TEXT	Kept for debugging and the write-up
attempts.density, accepted, reject_reasons	REAL, BOOLEAN, JSON	For the rejection rate
attempts.gen_ms, gen_tokens, tokens_per_sec, peak_memory_mb	REAL, INTEGER	LLM only; null for other arms

8.2 Indexes

- `keystroke_events` (`block_id`, `seq`) unique.
- `bigram_stats` (`participant_id`, `bigram`) and unique (`session_id`, `bigram`).

- drills (participant_id, arm, status) for picking the next ready drill.
- generation_jobs (status, created_at) for the worker poll.
- sessions (participant_id, phase, day_index).

9. UML: classes and states

9.1 Backend generation domain

Every generator implements the same DrillGenerator protocol. The GenerationService is the only class that talks to the database during generation, and the DrillValidator is the single fairness gate for all arms.



Figure 4. Backend class diagram for drill generation.

```

class Arm(StrEnum):
    CONTROL = "control"; HEURISTIC = "heuristic"; MARKOV = "markov"; LLM = "llm"

@dataclass(frozen=True)
class DrillSpec:
    participant_id: str
    arm: Arm
  
```

```

target_bigrams: tuple[str, ...] # empty for CONTROL
length_chars: int = 300
min_density: float = 0.15
max_density: float = 0.40
seed: int = 0

```

```

class DrillGenerator(Protocol):
    arm: Arm
    version: str
    def generate(self, spec: DrillSpec) -> DrillCandidate: ...

```

9.2 Frontend telemetry

The recorder is a plain TypeScript class. React subscribes to it with `useSyncExternalStore`, so a keystroke does not trigger a full re-render of the text.

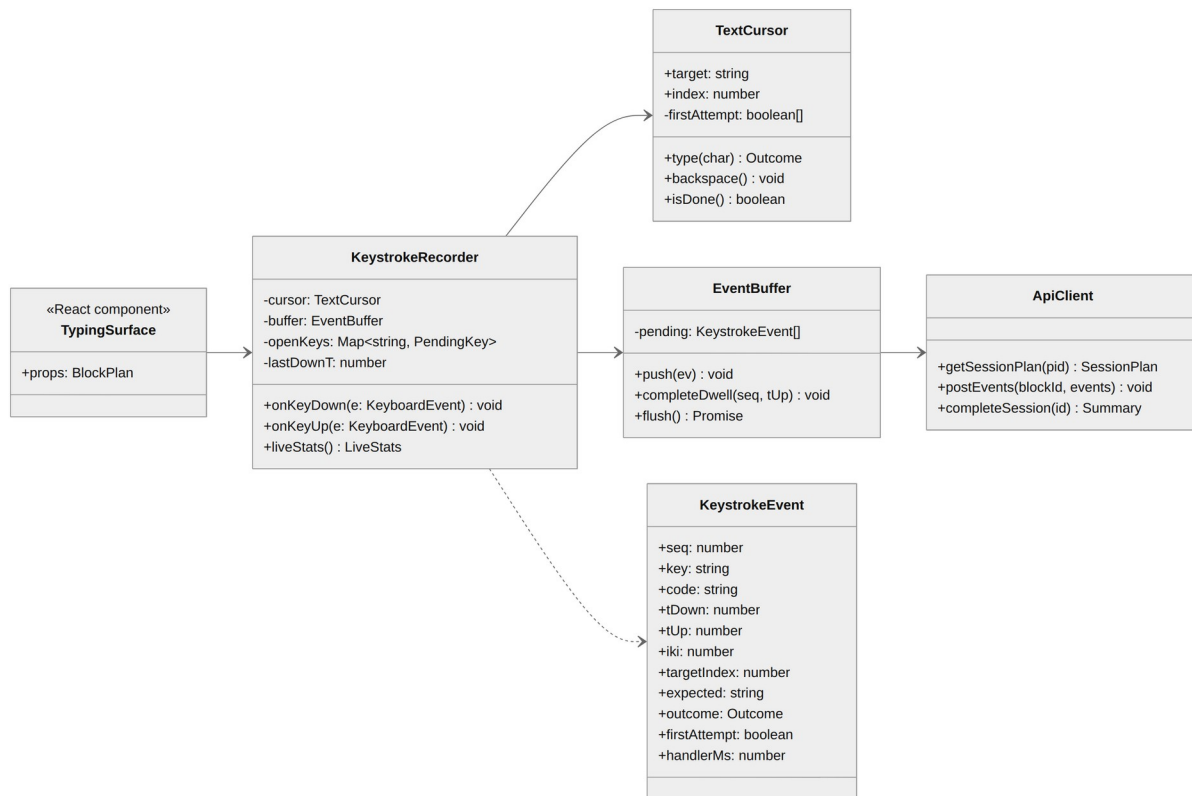


Figure 5. Frontend telemetry classes.

9.3 Participant lifecycle

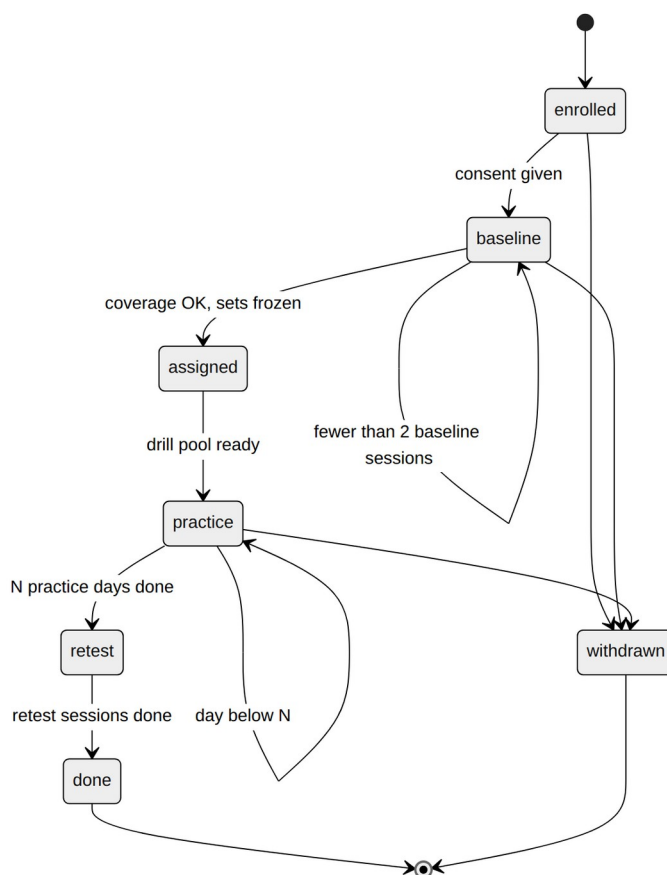


Figure 6. Participant state machine. Transitions are enforced in the services layer.

Sessions have their own small state machine: `in_progress` to `completed` when all blocks are finished, or to `abandoned` if a session is left unfinished for more than 30 minutes. Abandoned practice sessions do not count toward N, and their keystrokes are kept but excluded from outcome analysis.

10. Sequences and data flow

10.1 A typing session

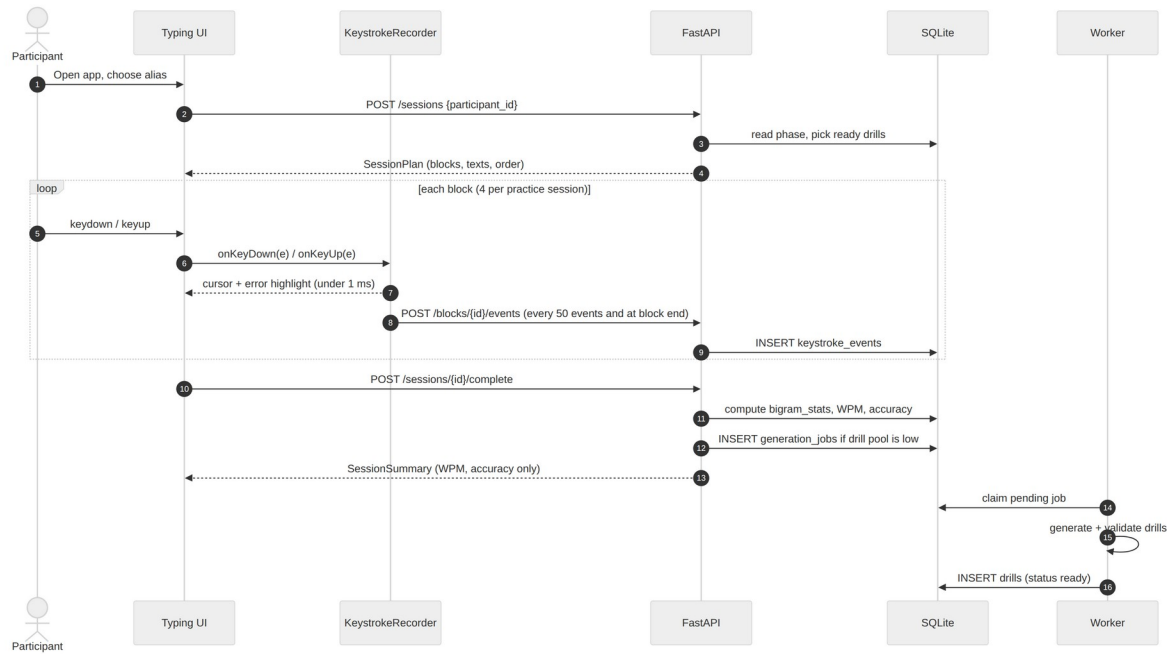


Figure 7. One practice session, end to end.

10.2 Drill generation job (LLM arm)

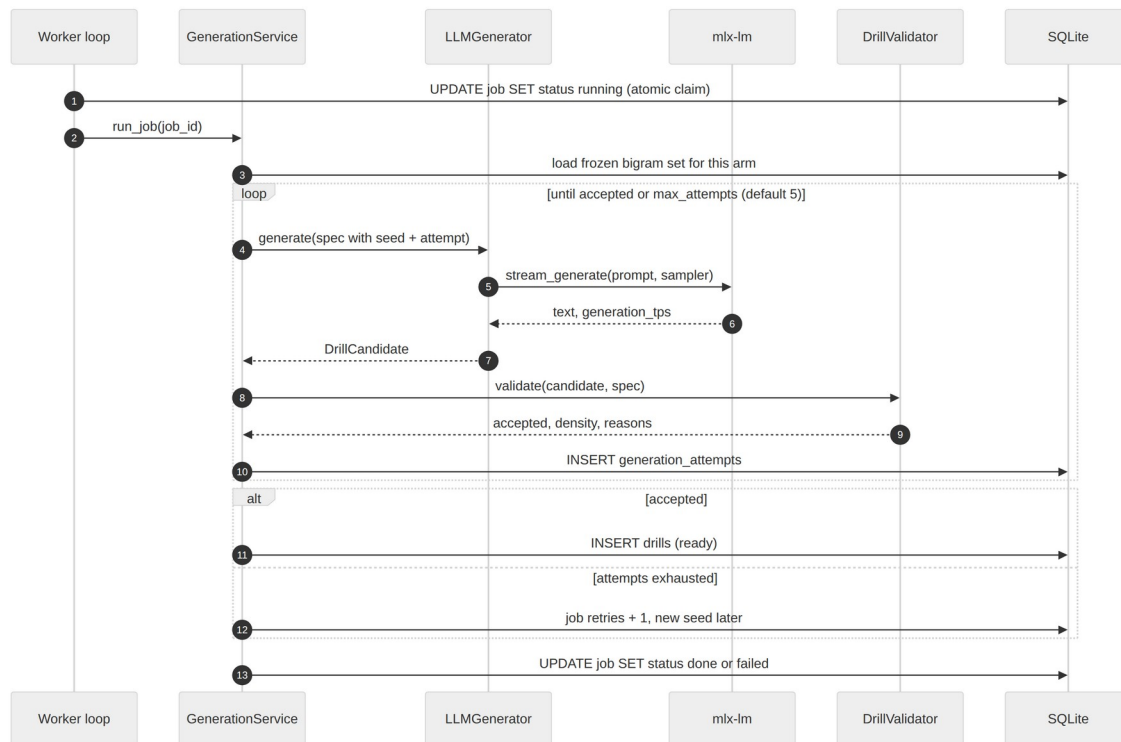


Figure 8. The worker's retry loop. Other arms follow the same path without the model call.

10.3 Data flow

Raw events become derived tables in one direction only. Anything to the right of `keystroke_events` can be deleted and rebuilt.

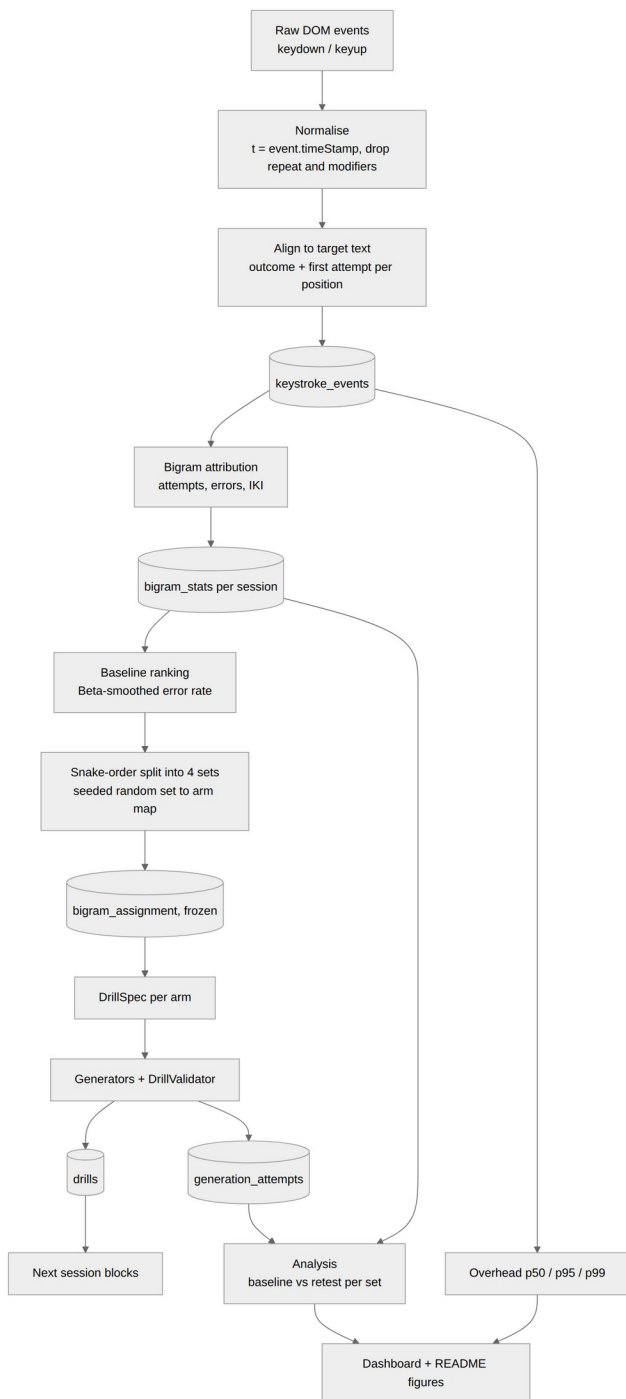


Figure 9. Data flow from DOM events to results.

11. User flows and screens

11.1 Participant flow

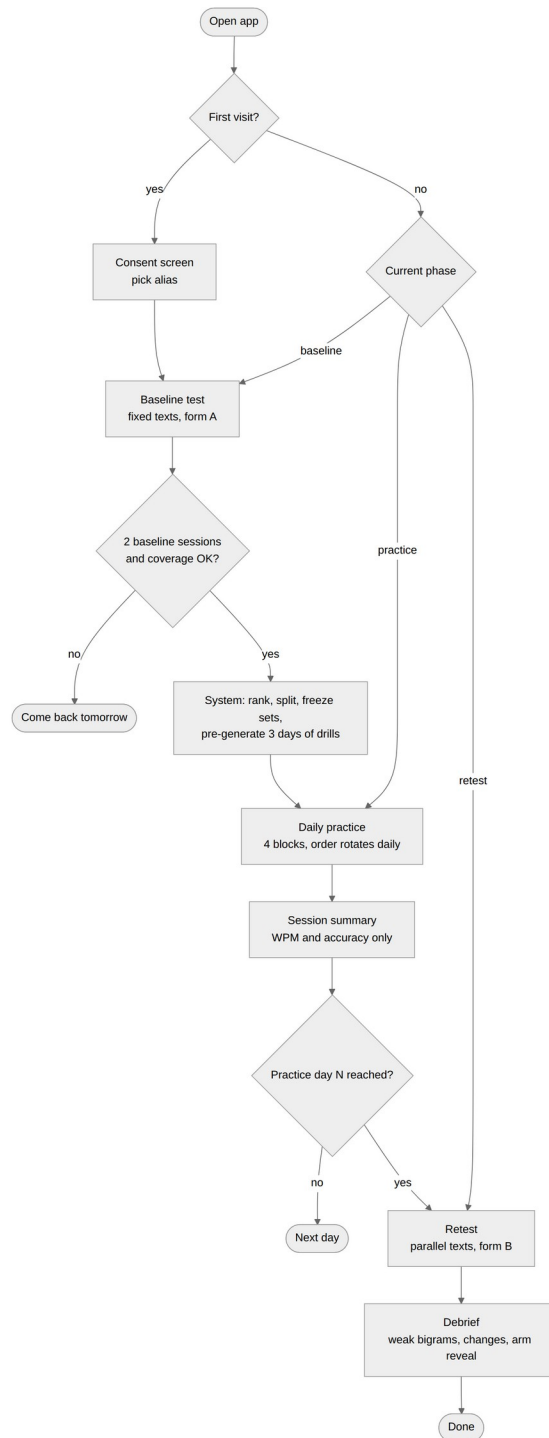


Figure 10. What a participant sees from first visit to debrief.

11.2 Builder (admin) flow

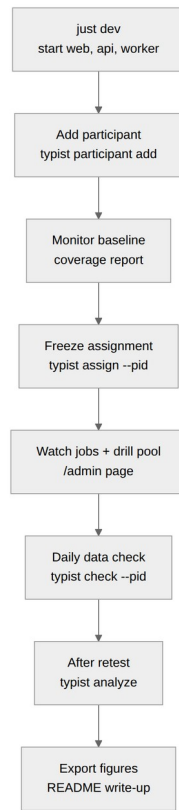


Figure 11. Builder tasks during the pilot.

11.3 Screens

Screen	Route	Purpose	Key elements
Participant picker	/	Choose alias or start as new	Alias list, new participant button
Consent	/consent	Explain what is recorded	Plain-language text, agree button
Session	/session/:id	Type the blocks	Block label, text with cursor, error highlight, progress bar, pause on window blur
Session summary	/session/:id/done	Close the session	WPM, accuracy, days remaining (no bigram details during the pilot)
Debrief	/results/:pid	After retest only	Weak bigrams, baseline versus retest per set, arm reveal
Admin	/admin	Builder tools	Participants and phases, drill pool per arm, job queue, data checks
Dashboard	/dashboard	Results	Per-set error and IKI change, WPM trend, overhead, LLM stats

Typing surface rules: monospace font, the whole block visible, one character per keystroke, wrong characters shown in red and left in place until corrected with Backspace, no autocorrect or spellcheck, and the block pauses if the window loses focus.

12. Telemetry specification

This is the most correctness-sensitive part of the project. If a keystroke is attributed to the wrong bigram, every result downstream is wrong. Build it test-first against the golden fixture.

12.1 Capture rules

- Listen to keydown and keyup on window while a block is active and the page has focus.
- Ignore events where `e.repeat` is true, modifier-only keys, and any event with Ctrl, Meta or Alt held. Shift is allowed but not needed in v1.
- Handle only a to z, space and Backspace. Call `preventDefault()` on handled keys so the browser does not scroll or type elsewhere.
- Use `e.key` to decide the character and store `e.code` for the physical key.
- Pair keyup with its keydown by `e.code`, using a map of open keys. This is what makes dwell correct when keys overlap.
- On window blur: pause the block, mark the next keystroke as following a pause, and resume on focus.

12.2 Timing definitions

Metric	Formula	Notes
Timestamp	<code>e.timeStamp</code>	High-resolution time when the event was created, same time origin as <code>performance.now()</code> . Do not use <code>Date.now()</code> . Browsers coarsen timers slightly for security; that is negligible for intervals of 50 to 300 ms, but record the browser and do not switch browsers mid-study
Dwell	<code>t_up</code> minus <code>t_down</code> , same physical key	How long a key is held
Inter-key interval (IKI)	<code>t_down(i)</code> minus <code>t_down(i-1)</code>	Primary timing measure (the guide's flight time). Always positive
Release to press	<code>t_down(i)</code> minus <code>t_up(i-1)</code>	Stored but secondary: it is negative whenever the next key is pressed before the previous one is released (rollover), which large typing studies found to be common
Pause	IKI over 2000 ms, or first key after a blur	Excluded from IKI statistics, still counted for errors

12.3 Alignment and bigram attribution

The `TextCursor` holds the target text and an index. Each printable keystroke is compared with `target[index]`, recorded as correct or error, and the index moves forward one position either way. Backspace moves the index back. The first keystroke ever recorded at a position is its first attempt; retyping after a correction is not.

Attribution then runs in Python over the stored rows:

```
for each position j >= 1 where target[j-1] and target[j] are letters:
    a = first_attempt keystroke at j-1
    b = first_attempt keystroke at j
    if a is missing or b is missing:           skip
    if a.outcome != "correct":                 skip # context was wrong
    if a.backspace happened between a and b:  skip # not a clean transition
```

```

bigram = target[j-1] + target[j]
attempts[bigram] += 1
if b.outcome == "error": errors[bigram] += 1
elif b.seq == a.seq + 1 and not b.after_pause:
    iki_samples[bigram].append(b.t_down - a.t_down)

```

- Session WPM is correct first-attempt characters divided by 5, divided by minutes of active typing (pauses removed).
- Session accuracy is correct first attempts divided by all first attempts.
- Bigrams across a space (for example e then space) are not tracked in v1 (D13).

12.4 Upload and durability

- A row is created on keydown and sent once its keyup has arrived, or after 2 seconds without one (keyup stays null).
- The buffer flushes every 50 rows, every 5 seconds, and at block end. Failed requests retry with backoff.
- The server inserts with ON CONFLICT (block_id, seq) DO NOTHING, so retries never duplicate rows.
- On page unload, the remaining rows are sent with `navigator.sendBeacon`.

12.5 Measuring instrumentation overhead

- `handler_ms`: `performance.now()` at the end of the handler minus at its start. This is the cost the recorder adds to each keystroke.
- `dispatch_lag_ms`: `performance.now()` at handler start minus `e.timeStamp`. This shows whether the main thread is busy (for example with rendering).
- Report p50, p95 and p99 per session with `typist` overhead. Target: handler p95 under 1 ms. The resume bullet uses the measured number, not the target.
- Keep the handler cheap: no React state updates inside it, no allocation-heavy work, no network calls. Rendering happens through the external store.

12.6 Daily data checks (typist check)

- Gaps in seq within a block.
- Negative dwell, or dwell over 1000 ms (stuck key).
- Blocks with fewer events than characters in the target text.
- Share of keyups missing.
- Sessions left in_progress.

13. Drill generation and model setup

13.1 Shared rules for every arm

Rule	Default	Applies to
Length	300 characters, cut at a word boundary; reject if under 270	All arms
Charset	Lowercase a to z and single spaces only	All arms
Target density	Target bigram occurrences divided by all letter-letter bigrams, between d_min 0.15 and d_max 0.40 (calibrated in M3, then frozen)	Heuristic, Markov, LLM
Coverage	Each target bigram appears at least 3 times	Heuristic, Markov, LLM
Repetition	No word more than 3 times; no bigram repeated 3 times in a row (for example thth th)	All arms
Real-word ratio	Share of tokens found in the word list, measured before cleaning; recorded, and LLM output under 0.8 is rejected	Recorded for all; gate for LLM
Determinism	Same spec and seed gives the same text (LLM: same seed and pinned versions)	All arms

Why the gate matters: the question is whether the method of choosing text matters. If one arm's text were much denser or longer, that alone could explain a difference. The validator forces all targeted arms into the same range, and the density, word length and real-word ratio of every drill are stored so the write-up can show the arms were comparable.

13.2 Control generator

Samples words from the 5,000 most frequent English words, weighted by the square root of frequency so very common words do not dominate. It ignores target bigrams. Its natural density for each set is recorded, which also shows how much more exposure the targeted arms gave.

13.3 Heuristic generator

1. At corpus build time, index the 20,000 most frequent a to z words by the letter bigrams they contain.
2. Keep a coverage counter for each target bigram.
3. Pick the target with the lowest coverage (ties broken by the seeded random generator).
4. Score candidate words containing it: score = target-bigram density of the word times $\log(\text{frequency})$. Sample one of the top 50 by softmax with temperature 0.5.
5. Skip words already used twice. Append, update coverage for every target in the word, and repeat until the length is reached.

13.4 Markov generator

1. Train a character-level model of order 3 on the same 20,000 words, with start and end markers, each word weighted by the square root of its frequency.
2. When sampling, if the previous character is the first letter of a target bigram, multiply the probability of that bigram's second letter by a bias factor beta (default 4), then renormalise.
3. Generate pseudo-words of 3 to 10 characters until the length is reached.
4. Tune beta in M3 so the median density lands inside the shared range, then freeze it.

Markov output is mostly pseudo-words. That is part of what the arm is, and it is reported as a difference between arms, not hidden.

13.5 LLM generator

Model choice

Candidate	Why consider it	Notes
mlx-community/Llama-3.2-1B-Instruct-4bit (default)	Matches the guide's ~1B framing; text-only; loads directly in mlx-lm	Released September 2024, so older than newer small families, which is acceptable for this question
mlx-community/Qwen3-1.7B-4bit	Newer and stronger at a similar size; text-only in mlx-lm	Disable its thinking mode for this task
Qwen3.5-0.8B or 2B, Gemma 4 E2B	Newest small families (2026)	Optional. Published MLX builds are multimodal and converted with mlx-vlm; include only if a text model loads in mlx-lm without extra work. Gemma 4 E2B needs about 5 GB at 4-bit

Runtime wrapper

```
import mlx.core as mx
from mlx_lm import load, stream_generate
from mlx_lm.sample_utils import make_sampler

class MLXRuntime:
    def __init__(self, model_id: str):
        self.model_id = model_id
        self.model, self.tokenizer = load(model_id)  # once per worker process

    def complete(self, messages, *, max_tokens=160, temp=0.8, top_p=0.9, seed=0):
        mx.random.seed(seed)
        prompt = self.tokenizer.apply_chat_template(
            messages, add_generation_prompt=True, tokenize=False)
        sampler = make_sampler(temp=temp, top_p=top_p)
        text, last = "", None
        for resp in stream_generate(self.model, self.tokenizer, prompt,
                                   max_tokens=max_tokens, sampler=sampler):
            text += resp.text
            last = resp
        return LLMOutput(text=text,
                          tokens=last.generation_tokens,
                          tokens_per_sec=last.generation_tps,
                          peak_memory_gb=last.peak_memory)
```

Check at milestone 0: confirm these function names and response fields against the installed mlx-lm version (the 0.31.x line was current in mid-2026), then pin that version. If a name differs, change the wrapper, not the callers.

Prompt v1

```
SYSTEM:
You write typing practice text. Reply with one line of lowercase English
```

words separated by single spaces. No punctuation, numbers, quotes or explanations.

USER (example turn):

Write 60 common English words. Many words must contain these letter pairs: ck, ow. Each pair should appear at least 3 times. Real words only.

ASSISTANT (example turn):

back know clock snow check brown quick below track show neck grow ...

USER:

Write 60 common English words. Many words must contain these letter pairs: {bigrams}. Each pair should appear at least 3 times. Real words only.

Setting	Value
Sampler	temperature 0.8, top_p 0.9, max_tokens 160
Seed	Derived from (participant, arm, drill number, attempt), stored on each attempt
Post-processing	Lowercase, replace anything outside a to z with a space, collapse spaces, measure real-word ratio, drop words not in the 50,000-word list, cut to length
Retries	Up to 5 attempts per drill with new seeds; every attempt is logged in generation_attempts
No fallback	If all attempts fail, the job fails and is retried later. The LLM arm never borrows heuristic text
Frozen for the pilot	model_id, prompt_version, sampler settings, d_min and d_max

13.6 Model bake-off (milestone 3, about half a day)

1. Make 10 synthetic target sets of 4 bigrams each (mix of common and awkward pairs).
2. For each candidate model, generate 20 drills per set through the real validator.
3. Record acceptance rate, attempts per accepted drill, density, real-word ratio, tokens per second and peak memory in notebooks/02_model_bakeoff.
4. Pick the model with the highest acceptance rate that runs at 30 tokens per second or more. If it is tied, keep Llama-3.2-1B.
5. Write the choice and numbers into an ADR and freeze the config. The prompt does not change after this.

13.7 Drill pool

- Target: 3 days of ready drills per arm per participant (2 drills per block times 3 days = 6).
- After each session completes, the API enqueues a job for any arm whose pool is below target.
- Drills are used once, in creation order, and marked used with the block id.

Deferred idea (v2): constrained decoding can force a model's output to match a pattern or grammar by masking tokens during generation. Libraries such as Outlines and lguidance can do this with mlx-lm through logits processors, but mlx-lm's built-in server does not apply a response format. It would be an interesting fifth arm ("LLM plus constraints") later, but it changes what the LLM arm means, so it is out of scope for this pilot.

14. API specification

All endpoints are JSON over `http://127.0.0.1:8000/api/v1`. FastAPI generates the OpenAPI schema; the frontend generates its types from it with just `gen-types`.

Method	Path	Purpose	Returns
GET	/health	Liveness, DB check, worker heartbeat age	status
GET / POST	/participants	List or create (alias, is_builder)	Participant
POST	/participants/{pid}/consent	Record consent, move to baseline	Participant
POST	/participants/{pid}/sessions	Start the next session for the current phase (returns the open one if it exists)	SessionPlan with blocks and texts
GET	/sessions/{sid}	Session with blocks	SessionPlan
POST	/blocks/{bid}/start	Mark block start time	Block
POST	/blocks/{bid}/events	Batch of keystroke rows (idempotent)	accepted and duplicate counts
POST	/blocks/{bid}/complete	Mark block end	Block
POST	/sessions/{sid}/complete	Compute stats, update phase, enqueue jobs	SessionSummary
POST	/admin/participants/{pid}/assign	Rank, split, map and freeze (only in baseline-done state)	Assignment
GET	/admin/pool	Ready drills per participant and arm	Pool
GET / POST	/admin/jobs	List jobs or enqueue manually	Jobs
GET	/admin/checks/{pid}	Data-quality report	Checks
GET	/analysis/participants/{pid}	Per-set outcomes with intervals	ParticipantResults
GET	/analysis/overview	All participants, overhead, generation stats	Overview

Errors use FastAPI's standard shape with a clear detail. Invalid state transitions (for example starting practice before assignment) return 409.

14.1 CLI (typist)

Command	Does
typist db migrate	Run Alembic migrations
typist corpus build	Build word index and n-gram model into data/
typist participant add --alias X [--builder]	Create a participant
typist participant delete --pid	Remove all of a participant's data
typist assign --pid	Same as the admin endpoint
typist worker	Run the generation worker
typist check --pid	Daily data-quality report

Command	Does
typist stats recompute [--pid]	Rebuild bigram_stats from raw keystrokes
typist overhead	Instrumentation overhead percentiles
typist analyze [--pid]	Outcomes, intervals and figures
typist export --out file	CSV or Parquet export for notebooks

15. Analysis and dashboard

15.1 Analysis plan (write before the pilot)

- Primary: targeting effect on error rate for heuristic, Markov and LLM sets, each versus the control set (section 3.5).
- Key secondary: the same for median IKI.
- Exclusions: abandoned sessions, pauses, bigrams with fewer than 20 retest attempts (reported, not silently dropped).
- Everything else is exploratory and labelled as such.
- Commit this plan with a date before the first practice session (D20).

15.2 Dashboard views

View	Shows
Bigram matrix (milestone 1)	26 by 26 heatmap of error rate and median IKI for one session or phase
Per-set outcomes	Baseline and retest error rate per set with intervals; effect versus control
Timing	Same for median IKI
Trend	WPM and accuracy per session, plus per-set error rate on practice text
Generation	Acceptance rate, attempts per drill, tokens per second, density per arm
Overhead	Handler and dispatch lag percentiles per session

15.3 README figures

- One chart: per-set error rate change with intervals, control set marked.
- One table: arm, drills, mean density, real-word ratio, acceptance rate.
- One line: overhead p95 and LLM tokens per second, measured.
- A limitations paragraph that states N, sessions and keystrokes.

16. Milestones and tasks

Estimates are in focused working days, not calendar days, because the build happens alongside the co-op. The pilot takes about 14 calendar days and overlaps milestone 4.

Milestone	Focused days	Done when	If work stops here
M0 Setup	1	just dev runs web, API and worker; CI green; model smoke test prints tokens per second on the MacBook	Nothing to show yet
M1 Telemetry and matrix	4	A baseline session produces a bigram matrix; overhead reported; recompute gives identical stats	Working diagnostic tool
M2 Loop with heuristic and control	4	A test participant goes baseline, assign, practice, retest end to end	Shippable project with a real experiment (2-set mode)
M3 Markov, LLM, bake-off, freeze	4	All four arms produce valid drills; config and analysis plan frozen	Full four-arm comparison ready
Pilot	about 14 calendar days	Both participants complete retest	Real numbers exist
M4 Dashboard and write-up	3	Dashboard and README with measured numbers	Presentable project

Changed cut-line: in the guide, later arms could be added during the pilot. In the within-person design the sets are split once, so every arm must exist before assignment. If M3 slips, run the pilot in 2-set mode (control and heuristic, 8 bigrams each), which the config supports with `arms_enabled`. That is still a real experiment and has more attempts per set.

M0 Setup

ID	Task	Acceptance criteria
T0.1	Create repository, justfile, pre-commit, .gitignore (data/)	<code>just --list</code> shows dev, test, lint, types
T0.2	Scaffold apps/web: Vite 8, React 19, TS strict, Biome, Vitest, Tailwind	<code>just test</code> runs one passing web test
T0.3	Scaffold services/api: uv, FastAPI, SQLAlchemy, Alembic, ruff, pyright, pytest; /health	<code>curl /api/v1/health</code> returns ok
T0.4	GitHub Actions: lint, types, tests for both apps	CI green on main
T0.5	Model smoke test: load Llama-3.2-1B-Instruct-4bit with mlx-lm and generate one line	Prints text and tokens per second; versions recorded
T0.6	Create docs/analysis-plan.md skeleton	File exists with sections from 15.1

M1 Telemetry and matrix

ID	Task	Acceptance criteria
T1.1	Tables and migration: participants, experiment_config, text_passages, sessions, session_blocks, keystroke_events	Migration up and down both work

ID	Task	Acceptance criteria
T1.2	Golden fixture: hand-checked keystroke log with expected outcomes and bigram stats	Reviewed line by line; includes errors, backspace, rollover, a pause
T1.3	TextCursor with unit tests against the fixture	All fixture outcomes match
T1.4	KeystrokeRecorder: filters, timestamps, dwell pairing by code, IKI, overhead fields	Tests with synthetic KeyboardEvents cover repeat, modifiers, overlapping keys
T1.5	TypingSurface with useSyncExternalStore; pause on blur	Typing 300 characters feels instant; no full re-render per key
T1.6	EventBuffer and idempotent events endpoint	Sending the same batch twice stores it once
T1.7	attribution.py with fixture tests and a hypothesis property test	Fixture matches; property: attempts never exceed letter-pair positions
T1.8	Corpus build and baseline text composer (form A)	Two baseline texts give 15 or more occurrences for the top ~150 bigrams
T1.9	Session completion: bigram_stats, WPM, accuracy	typist stats recompute reproduces the same numbers
T1.10	Overhead report and bigram matrix page	p50, p95 and p99 printed; heatmap renders

M2 Loop with heuristic and control

ID	Task	Acceptance criteria
T2.1	Tables: bigram_stats, bigram_assignment, drills, generation_jobs, generation_attempts	Migration works
T2.2	assignment.py: smoothing, ranking, snake split, seeded mapping, freeze	Same seed gives same mapping; frozen rows cannot be updated
T2.3	generation/base.py and validator.py with tests	Each rejection reason has a test
T2.4	Control and heuristic generators	50 drills each pass the validator; deterministic by seed
T2.5	Worker loop and GenerationService	Atomic job claim; failures logged; heartbeat visible on /health
T2.6	Session planner: Latin square, pool selection, marking drills used	Block order rotates daily; no drill used twice
T2.7	Retest composer (form B)	At least 40 attempts per tracked bigram; different words from form A where possible
T2.8	Participant state transitions	Invalid transitions return 409
T2.9	Admin page: participants, pool, jobs, checks	Builder can run the pilot without the terminal except just dev
T2.10	Dev-only pipeline test using scripted keystrokes	Full flow passes in CI. Never used as evidence (the guide dropped simulated typists)

M3 Markov, LLM, bake-off, freeze

ID	Task	Acceptance criteria
T3.1	Character n-gram model and Markov generator; tune beta	Median density inside the shared range

ID	Task	Acceptance criteria
T3.2	MLXRuntime, prompt v1, LLM generator	Drills pass the validator; attempts logged with tokens per second
T3.3	Bake-off notebook (section 13.6)	Table of results and an ADR with the choice
T3.4	Calibrate d_min and d_max so all targeted arms can reach them	Each targeted arm accepts 80% or more of drills within 5 attempts
T3.5	Freeze experiment_config	Frozen keys reject changes
T3.6	Commit the dated analysis plan	Commit exists before the first practice session

Pilot runbook

- Day 0 and 1: both participants do the two baseline sessions. Run `typist` check each evening.
- End of day 1: run `assign` for each participant; confirm the pool has 3 days of drills per arm.
- Days 2 to 11: one practice session per day each. Watch the admin page for failed jobs.
- Day 12 and 13: retest sessions, then debrief.
- Do not change code that affects telemetry, generation or text composition during the pilot. Bug fixes that do are logged as deviations in the write-up.

M4 Dashboard and write-up

ID	Task	Acceptance criteria
T4.1	analysis.py: outcomes, Beta intervals, posterior effect samples	Unit tests on small hand-computed examples
T4.2	Dashboard pages (section 15.2)	All views load from real data
T4.3	Results notebook and README figures	Figures regenerate from one command
T4.4	README with numbers, method, limitations, N	Every claim traces to a number in the database
T4.5	Debrief page	Shows arm mapping only after retest

17. Engineering standards

17.1 Code

- TypeScript strict: true, no any without a comment explaining why. Python type hints everywhere, pyright in strict mode for src/.
- Routers are thin. Business logic lives in services. Table models are never returned from the API; use response schemas.
- Every experiment parameter lives in config, never as a literal inside logic.
- Functions do one thing. If a function needs a comment to explain what it does, split or rename it.
- Randomness always goes through a seeded generator passed in from the caller.

17.2 Tests

Area	What to test	Type
TextCursor, recorder	Outcomes, dwell pairing, filters, overhead fields	Vitest unit
Attribution	Golden fixture, edge cases, property tests	pytest + hypothesis
Assignment	Ranking, snake order, seed determinism, freeze	pytest unit
Validator and generators	Each rule, determinism, density range	pytest unit
API	Endpoints and state transitions on a temporary SQLite file	pytest integration with httpx
End to end	Type a short block in a real browser, check rows stored	Playwright smoke

17.3 Workflow

- Branch per ticket (t1.3-text-cursor), conventional commits, merge through a pull request with CI green.
- Any change to a decision gets a short ADR in docs/decisions/.
- Use AI coding tools freely, but review generated code in telemetry and attribution line by line against the fixture.

18. Risks

Risk	Impact	Mitigation
Only 1 to 2 participants	Results describe these people only	Within-person design; state N plainly; no significance claims
Few errors per bigram	Noisy primary outcome	Pool by set; at least 40 attempts per bigram at retest; IKI as key secondary
Spill-over between sets	Effects look smaller	Report as a limitation; it biases toward no difference, not toward a false win
Builder not blind	Expectation effects in own data	Flag builder data; show the second participant separately
Attribution bug	All results wrong	Golden fixture shared by both languages; recompute command; property tests
1B model writes poor drills	LLM arm starved of drills	Retries, bake-off, 3-day pool; report rejection rate
Markov pseudo-words	Difficulty differs between arms	Record word length and real-word ratio; discuss
Scope creep alongside the co-op	Pilot never starts	Cut-line to 2-set mode; M1 alone is a working tool
Second participant drops out	Only builder data	Still a valid within-person result for one person; say so
Library changes mid-project	Behaviour drift	Lockfiles; no upgrades between freeze and retest

19. Research notes and resources

19.1 What the tool research found (September 2026)

- **mlx-lm** is Apple's Python package for running and fine-tuning language models on Apple Silicon; the 0.31.x line was current in mid-2026, with a new release every few weeks. It is the most direct way to get sampler control and generation speed statistics on the M1 Pro.
- **Ollama** now uses MLX under the hood on Apple Silicon and is easier to install, but gives less control over seeds and sampling from Python. It is a reasonable fallback, not the default.
- **Newer small models exist.** Gemma 4 (April 2026) has E2B and E4B edge sizes, and Qwen3.5 (March 2026) has 0.8B and 2B sizes. Their published MLX builds are multimodal and mostly converted with mlx-vlm, and Gemma 4 E2B needs around 5 GB at 4-bit, so they are optional bake-off entries rather than the default.
- **Llama 3.2 1B** (September 2024) is no longer new, but it is text-only, small and well supported in mlx-lm, and it matches the guide's framing.
- **Constrained decoding** is possible with mlx-lm through logits processors (Outlines, llguidance), but the built-in mlx-lm server does not apply response formats. Deferred to v2.
- **Typing research:** a large study of 136 million keystrokes (Dhakal et al., CHI 2018) found rollover typing is common and that letter pairs typed with different hands or fingers predict speed. That supports using press-to-press intervals and suggests weak bigrams will often be same-hand or same-finger pairs. The dataset is free for research use and could be used later to sanity-check your baseline.
- **Vite 8** (March 2026) replaced esbuild and Rollup with the Rust-based Rolldown bundler; @vitejs/plugin-react v6 no longer needs Babel.

19.2 Links

Resource	Use
github.com/ml-explore/mlx-lm	Model runtime docs and examples
huggingface.co/mlx-community/Llama-3.2-1B-Instruct-4bit	Default model
ai.google.dev/gemma/docs/core	Gemma 4 sizes and memory
dottxt-ai.github.io/outlines (mlx-lm integration)	Constrained decoding, for v2
userinterfaces.aalto.fi/136Mkeystrokes	Dhakal et al. keystroke dataset and paper
developer.mozilla.org : KeyboardEvent, Event.timeStamp, performance.now	Event and timing reference
vite.dev/blog/announcing-vite8	Vite 8 changes
fastapi.tiangolo.com , sqlmodel.tiangolo.com , docs.astral.sh/uv	Backend docs
github.com/rspeer/wordfreq	Word frequencies (cite the data license in the README)
github.com/monkeytypegame/monkeytype	UI reference only; not forked

20. Open items and day-one checklist

20.1 Open items (none block milestone 0)

Item	Default until decided	Decide by
Who the second participant is	Builder only	Before baseline
Remote access for them (option B)	Same laptop	Before baseline
Practice days N	10	Freeze at M3
Density range	0.15 to 0.40	Calibrated at M3
Final model	Llama-3.2-1B-Instruct-4bit	Bake-off at M3

20.2 After the pilot (v2 ideas)

- Live nudges when a known weak bigram is mistyped.
- Trigrams, punctuation, capitals and code text.
- Constrained-decoding LLM arm.
- More participants and a between-person design.

20.3 Day-one checklist

1. Install uv, pnpm, just, Node 22 and Python 3.12 (uv can install Python).
2. Create the repository with the layout in section 7.
3. Do T0.5 first: prove the model runs on the MacBook and record the versions.
4. Finish the rest of M0, then write the golden fixture (T1.2) before any telemetry code.